

Universidad Gerardo Barrios
Facultad de Ciencia y Tecnología

Informe final de proyecto

Trabajo de graduación — Modalidad preespecialización en la
especialidad de:

Desarrollo de Aplicaciones con Inteligencia Artificial

“VeriNewsIA & Phishing”

Extensión de navegador con backend de IA para la detección de
phishing en tiempo real

Presentado por:

Melvin Santiago Rodríguez Mendoza

Katia Guadalupe Orellana Romero

Eskin Onan Montiel Hernández

San Miguel, 22 de agosto de 2026

Índice

Resumen del proyecto:	3
Marco conceptual:	4
1. Phishing y detección automatizada de URLs	4
2. Aprendizaje automático aplicado a la clasificación de URLs	6
2.1. Random Forest	6
2.2. Transformers y RoBERTa	7
2.3. Fusión de modelos (ensembling)	8
3. Motores de reglas explicables (scoring aditivo)	9
4. Inteligencia de amenazas (Threat Intelligence)	10
5. Arquitectura de API REST con FastAPI	11
6. Seguridad de aplicaciones web expuestas a entrada no confiable	12
7. Extensiones de navegador (Manifest V3)	13
8. Prácticas de ingeniería: contenedores, CI/CD y observabilidad	14
Análisis del problema:	15
Propuesta de solución:	16
Metodología de desarrollo del proyecto:	18
1. API inteligente y contratos	18
2. Pruebas y CI/CD	19
3. Despliegue e infraestructura	20
Resultados alcanzados	20
1. Observabilidad, rendimiento y escalabilidad	20
2. Seguridad, release y rollback	22
Conclusiones:	23
Referencias:	24

Resumen del proyecto:

VeriNewsIA & Phishing es una extensión para navegadores basados en Chromium (Google Chrome, Microsoft Edge), construida sobre Manifest V3, que permite a cualquier usuario verificar en tiempo real si una URL representa un riesgo de phishing antes de introducir credenciales o datos sensibles en ella. El problema que atiende es concreto: el phishing sigue siendo uno de los vectores de ataque más usados para el robo de credenciales, y la mayoría de los usuarios no cuenta con una herramienta rápida, explicable y gratuita para evaluar un enlace sospechoso antes de interactuar con él.

El usuario objetivo es la persona que navega activamente por internet (correo, redes sociales, resultados de búsqueda) y necesita una segunda opinión inmediata sobre un enlace antes de decidir si confiar en él sin conocimientos técnicos previos, y sin instalar software adicional más allá de la extensión.

La función de la inteligencia artificial dentro del producto es doble. Por un lado, dos modelos de Machine Learning entrenados específicamente para este dominio, un Random Forest sobre 34 características extraídas de la URL y el HTML, y un modelo RoBERTa afinado (fine-tuned) sobre el texto crudo de la URL, se combinan mediante una fusión ponderada (40 % / 60 %) para producir una probabilidad de phishing. Por otro lado, un tercer modelo RoBERTa, especializado en clasificación de contenido (noticias reales vs. falsas), está disponible como servicio independiente para evaluar el texto de una página. Estas señales de IA se combinan con reglas heurísticas explícitas (antigüedad del dominio, presencia de IP en vez de dominio, símbolos de engaño, TLD sospechosos) y con tres fuentes externas de inteligencia de amenazas (VirusTotal, Google Safe Browsing y Google Fact Check Tools) dentro de un motor de riesgo aditivo y auditable, que entrega un puntaje de 0 a 100, un veredicto LOW / MEDIUM / HIGH y una lista de razones en lenguaje natural que explican la decisión.

La propuesta de valor frente a alternativas existentes es la combinación de explicabilidad (cada veredicto se acompaña de sus razones y de una confianza calibrada), degradación controlada (si una API externa o un modelo falla, el resto

de las señales sigue funcionando en vez de tumbar el análisis completo) y un pipeline paralelo que entrega una respuesta completa en 3 a 8 segundos. El resultado del proyecto durante las seis sesiones del módulo es un backend en FastAPI con 11 endpoints documentados, una batería de 535 pruebas automatizadas, un pipeline de integración continua en GitHub Actions, una imagen Docker desplegable y una extensión de navegador completa (popup, panel lateral y página de configuración), junto con un ciclo de retroalimentación del usuario que sienta la base para el reentrenamiento futuro de los modelos con datos reales.

Marco conceptual:

1. Phishing y detección automatizada de URLs

El phishing es una técnica de ingeniería social en la que un atacante suplanta la identidad de una entidad confiable (un banco, una red social, un servicio de correo) para engañar a la víctima y obtener credenciales, datos financieros o acceso a sistemas. A diferencia del malware tradicional, el phishing no depende de explotar una vulnerabilidad de software: depende de que el usuario tome una decisión de confianza equivocada frente a una URL, un correo o una página que imita a la legítima. Por eso la mayoría de las defensas automatizadas — incluida la de este proyecto — se concentran en analizar la URL y la página de destino antes de que el usuario interactúe con ella, en vez de intentar detectar la intención del atacante directamente.

Existen dos grandes familias de enfoques para esta detección. La primera es la basada en listas negras (blocklists): un dominio se marca como malicioso una vez que ha sido reportado, y los servicios de terceros (VirusTotal, Google Safe Browsing) distribuyen esa lista. Su ventaja es la precisión casi perfecta una vez que un sitio está catalogado; su debilidad es la ventana de tiempo antes de esa catalogación, en la que un dominio de phishing recién creado pasa inadvertido. La segunda familia es la basada en heurísticas y aprendizaje automático: en vez de esperar un reporte, se extraen características de la URL, del dominio y del contenido de la página, y un modelo — o un conjunto de reglas — estima la probabilidad de que se trate de un sitio fraudulento. Este proyecto combina ambos

enfoques deliberadamente: los modelos de ML y las heurísticas cubren la ventana temprana en la que un dominio aún no ha sido reportado, y las APIs de threat intelligence aportan certeza absoluta cuando el sitio ya es conocido.

El phishing no es una categoría única: la literatura de seguridad distingue variantes según el canal y el objetivo. El **spear phishing** se dirige a una persona o cargo específico con información personalizada que aumenta la credibilidad del engaño; el **whaling** es una variante de spear phishing enfocada en ejecutivos o figuras de alto perfil; el **smishing** y el **vishing** usan SMS y llamadas de voz en lugar de correo o web; el **clone phishing** duplica un mensaje legítimo previamente recibido y reemplaza sus enlaces adjuntos; y el **pharming** manipula la resolución DNS para redirigir al usuario sin que este haga clic en nada. Este proyecto se concentra deliberadamente en el vector más extendido — el enlace malicioso entregado por cualquier canal pero verificado en el navegador — porque es el punto de la cadena donde una extensión puede intervenir sin depender del canal de origen del ataque.

El ciclo de vida de un ataque de phishing basado en URL tiene una ventana temporal crítica: el atacante registra un dominio, monta una réplica de la página legítima, y distribuye el enlace antes de que cualquier servicio de reputación lo haya catalogado. Esa ventana — que puede ir de horas a pocos días — es precisamente donde las listas negras fallan y donde el análisis heurístico y los modelos de ML entrenados localmente aportan valor, porque no dependen de que alguien haya reportado el dominio primero. La ingeniería social que sostiene el engaño se apoya en gatillos psicológicos bien documentados — urgencia ("tu cuenta será suspendida"), autoridad (suplantar a un banco o una entidad gubernamental) y familiaridad visual (clonar el diseño exacto del sitio real) — que el análisis técnico de la URL y del HTML puede detectar aunque el contenido visual sea indistinguible a simple vista.

Dentro del análisis puramente léxico de la URL, la literatura de seguridad documenta patrones recurrentes que el proyecto traduce directamente en características de entrada para sus modelos: el typosquatting registra dominios que difieren del legítimo por un carácter ("paypa1.com", "gogle.com"), apostando a que el usuario no note el error; los ataques homográficos (IDN homograph attacks) van un paso más allá y sustituyen caracteres latinos por caracteres visualmente

idénticos de otros alfabetos (por ejemplo, una letra “a” cirílica en lugar de la latina), de forma que el dominio se ve exactamente igual al ojo humano aunque sea técnicamente distinto; y los acortadores de URL (“bit.ly”, “tinyurl.com” y equivalentes) ocultan el dominio de destino real detrás de una redirección, negando al usuario la posibilidad de inspeccionar la URL antes de hacer clic. Estos tres patrones son precisamente el tipo de señal que un extractor de características puramente léxico (`url_features.py`) puede capturar de forma determinista — longitud del dominio, cantidad de subdominios, presencia de caracteres no ASCII, uso de servicios de acortamiento — sin necesidad de descargar ni ejecutar el contenido de la página.

2. Aprendizaje automático aplicado a la clasificación de URLs

2.1. Random Forest

Un Random Forest es un modelo de ensamble compuesto por muchos árboles de decisión entrenados sobre subconjuntos aleatorios de los datos y de las variables (bagging), cuya predicción final es el promedio (o el voto mayoritario) de todos los árboles individuales. Se eligió para este proyecto porque trabaja de forma natural con datos tabulares heterogéneos — en este caso, 34 características numéricas y booleanas extraídas de la URL y del HTML de la página (longitud de la URL, cantidad de subdominios, presencia de un formulario de contraseña, cantidad de scripts, etc.) —, es comparativamente económico de entrenar e inferir frente a un modelo de lenguaje, y permite inspeccionar la importancia relativa de cada característica, lo que ayuda a auditar por qué el modelo decide lo que decide. El modelo entrenado (`random_forest_v2.pkl`, con su orden de columnas fijado en `feature_columns_v2.pkl`) usa la convención de etiquetas 0 = phishing / 1 = legítimo; la probabilidad de phishing se toma de la clase 0 y la de sitio legítimo de la clase 1.

Cada árbol individual crece dividiendo el espacio de características mediante preguntas binarias (por ejemplo, “¿la URL tiene más de 3 guiones?”) elegidas para maximizar la pureza de las dos ramas resultantes, medida con un criterio como la impureza de Gini o la entropía. Un solo árbol profundo tiende a sobreajustarse — memoriza el conjunto de entrenamiento en vez de generalizar —, pero el bagging

lo corrige de dos formas simultáneas: cada árbol ve solo una muestra aleatoria con reemplazo de los datos (bootstrap), y en cada división solo se le permite considerar un subconjunto aleatorio de las columnas. Eso hace que los árboles del bosque cometan errores distintos entre sí; al promediarlos, el error no correlacionado se cancela y la varianza total del modelo baja sin aumentar su sesgo. El modelo entrenado usa 300 árboles con una profundidad máxima de 29, lo que produce cerca de 321 000 nodos en total — un tamaño que se sostiene en memoria sin problema (unos 156 MB medidos), a diferencia de un modelo de lenguaje, donde profundizar la arquitectura sí tiene un costo de RAM importante. Además de predecir, un Random Forest entrenado con `scikit-learn` expone la importancia relativa de cada característica en la decisión final, lo que se usó durante el desarrollo como herramienta de auditoría: permite confirmar que el modelo se apoya en señales con sentido de dominio (edad del dominio, presencia de formularios de contraseña, cantidad de subdominios) y no en artefactos espurios del conjunto de entrenamiento.

2.2. Transformers y RoBERTa

RoBERTa es una variante optimizada de BERT, un modelo de lenguaje basado en la arquitectura Transformer, que aprende representaciones contextuales de texto mediante mecanismos de auto-atención (self-attention): a diferencia de un modelo tabular, no necesita que alguien defina a mano qué características observar, sino que aprende directamente patrones sobre la secuencia de caracteres o palabras. El proyecto usa dos instancias distintas de este modelo, cada una afinada (fine-tuning) sobre un corpus distinto: una versión ligera (`distilroberta-base`) fine-tuneada directamente sobre cadenas de URL para capturar patrones léxicos propios del phishing (subdominios engañosos, combinaciones de palabras como “verify”, “secure”, “login”, sustituciones de caracteres), y una segunda instancia fine-tuneada sobre un corpus de noticias reales y falsas (`GonzaloA/fake\_news`) para la clasificación de contenido, expuesta como servicio independiente.

El mecanismo de auto-atención es lo que distingue a un Transformer de arquitecturas secuenciales anteriores (como las redes recurrentes): en vez de procesar el texto token por token en orden, calcula para cada token un peso de relación con todos los demás tokens de la secuencia a la vez, lo que le permite

capturar dependencias de largo alcance (por ejemplo, relacionar un subdominio al inicio de la URL con un patrón sospechoso hacia el final) sin la degradación progresiva que sufren los modelos recurrentes. RoBERTa parte de la arquitectura de BERT pero cambia su procedimiento de preentrenamiento: se entrena sobre un corpus mucho mayor, aplica enmascaramiento dinámico (el patrón de palabras ocultas cambia en cada época en vez de fijarse una sola vez) y elimina la tarea de predicción de la siguiente oración (Next Sentence Prediction), que estudios posteriores a BERT mostraron que aportaba poco. Antes de llegar al modelo, el texto se descompone con tokenización por pares de bytes (Byte-Pair Encoding, BPE), que parte palabras poco frecuentes en subunidades reutilizables; esto es especialmente valioso para URLs, porque permite que el modelo reconozca fragmentos como “verify” o “secure” dentro de un dominio nunca visto durante el entrenamiento (por ejemplo, `verify-secure-bank2024.tk`), en vez de tratar el dominio completo como una palabra desconocida. El proyecto usa fine-tuning completo (se reajustan los pesos del modelo preentrenado sobre el dataset propio) en lugar de extracción de características con los pesos congelados, porque el vocabulario y la sintaxis de una URL difieren lo suficiente del lenguaje natural con el que RoBERTa fue preentrenado como para justificar el costo adicional de reentrenar toda la red.

2.3. Fusión de modelos (ensembling)

El `FusionEngine` combina la probabilidad de phishing del Random Forest y la del RoBERTa de URL con una ponderación fija (40 % / 60 % a favor de RoBERTa), en vez de promediar sin más. La justificación es que un modelo de lenguaje entrenado directamente sobre el texto de la URL captura mejor los patrones léxicos del phishing moderno que un conjunto fijo de características tabulares, aunque ambos aportan señal independiente. El componente degrada con elegancia: si uno de los dos modelos falla en tiempo de ejecución, el otro pasa a operar con el 100 % del peso en vez de propagar el error.

Combinar modelos (ensembling) es una técnica general con tres variantes principales: bagging (entrenar varios modelos independientes sobre muestras distintas de los datos y promediar sus salidas — el propio Random Forest es un caso de esto), boosting (entrenar modelos secuencialmente, cada uno corrigiendo

los errores del anterior) y stacking (entrenar un meta-modelo que aprende cómo combinar las salidas de varios modelos base). El proyecto usa una variante deliberadamente más simple que las tres: una ponderación fija decidida a mano, en vez de un meta-modelo aprendido. La razón no es de rendimiento sino de datos disponibles: entrenar un stacking requiere un conjunto de validación etiquetado independiente del usado para entrenar los modelos base, y ese conjunto no existe todavía para este proyecto (ver Conclusiones generales). La ponderación 40/60 es, por tanto, una decisión informada por la literatura y por pruebas manuales, no el resultado de una búsqueda de hiperparámetros — una limitación reconocida explícitamente y documentada como trabajo futuro.

3. Motores de reglas explicables (scoring aditivo)

En vez de delegar la decisión final íntegramente en un modelo de caja negra, el proyecto usa un motor de riesgo (``RiskEngine``) basado en un modelo aditivo: parte de un puntaje neutro de 50 sobre 100 y aplica penalizaciones o bonificaciones explícitas por cada señal detectada (HTTPS, edad del dominio, presencia de un formulario de contraseña, resultado de cada API externa, resultado de la fusión de modelos, etc.), hasta acotar el resultado al rango $[0, 100]$. Este diseño prioriza la explicabilidad (Explainable AI): cada veredicto se puede descomponer en la lista exacta de razones que lo produjeron, algo que un único modelo de caja negra no ofrece por sí solo. Sobre ese puntaje se añade además una capa de calibración (``confidence_calibration.py``) que traduce el score a una probabilidad mediante una función sigmoide y reporta un nivel de confianza explícito, que baja cuando los modelos discrepan entre sí o cuando pocas señales contribuyeron al resultado.

Esta decisión de diseño responde a una disyuntiva conocida en la literatura de Explainable AI (XAI): explicar un modelo por diseño (interpretable by design) frente a explicar un modelo de caja negra ya entrenado mediante técnicas post-hoc, como LIME o SHAP, que aproximan localmente el comportamiento del modelo para estimar qué características influyeron en una predicción puntual. Esas técnicas son valiosas pero producen una aproximación, no una explicación exacta, y añaden costo computacional en cada inferencia. El ``RiskEngine`` evita ese problema por construcción: como el puntaje se arma sumando deltas conocidos de antemano, la

explicación no se aproxima después del hecho, es literalmente la lista de operaciones que produjo el resultado. La calibración de confianza complementa esta explicabilidad con una segunda dimensión, distinta de “qué señales influyeron”: “cuánto se puede confiar en este veredicto en concreto”, que baja automáticamente cuando las distintas señales se contradicen entre sí, en vez de reportar siempre el mismo nivel de certeza independientemente de cuán conflictiva haya sido la evidencia.

Un ejemplo concreto ilustra el mecanismo: una URL arranca en 50 puntos; si usa HTTPS suma unos pocos puntos, si el dominio tiene más de un año de antigüedad suma otros, si el hostname contiene múltiples guiones (medido sobre ``num_hyphens_domain``, no sobre la URL completa, para no penalizar rutas largas de sitios legítimos como noticias o documentación) resta puntos, y una detección positiva en Google Safe Browsing resta hasta 50 puntos por sí sola. Cuando VirusTotal y Safe Browsing coinciden en que el sitio está limpio, la penalización por dominio joven se atenúa mediante un factor de amortiguación (``_YOUNG_DOMAIN_DAMPING``), porque un dominio recién registrado pero no reportado por ninguna fuente de reputación es, estadísticamente, más probable que sea un sitio legítimo nuevo que un ataque de phishing todavía no descubierto; esa amortiguación nunca se aplica si alguna fuente ya confirmó una amenaza. El resultado final se acota entre 0 y 100 y se traduce a tres niveles: 80 o más es riesgo bajo, entre 50 y 79 es riesgo medio, y por debajo de 50 es riesgo alto.

4. Inteligencia de amenazas (Threat Intelligence)

Las APIs de threat intelligence integradas — VirusTotal API v3, Google Safe Browsing API v4 y Google Fact Check Tools API — son bases de datos colaborativas mantenidas por terceros que agregan reportes de múltiples motores antivirus y de múltiples fuentes de verificación de datos. Su función en el pipeline no es predecir, sino confirmar: cuando un dominio ya fue reportado como malicioso, estas fuentes lo detectan con una precisión muy superior a la de cualquier modelo entrenado localmente, y por eso sus señales tienen el mayor peso negativo dentro del ``RiskEngine`` (hasta -50 puntos ante una detección de Safe Browsing).

La inteligencia de amenazas suele clasificarse en niveles según su nivel de abstracción: estratégico (tendencias de alto nivel para decisiones de negocio), táctico (tácticas y técnicas de atacantes), operacional (información sobre campañas concretas en curso) y técnico (indicadores de compromiso — IOCs — concretos y verificables, como un hash de archivo, una IP o un dominio reportado). Las tres integraciones del proyecto operan en este último nivel: consultan IOCs puntuales contra bases colaborativas ya construidas, en vez de generar inteligencia nueva. Esa naturaleza colaborativa también implica un compromiso conocido entre falsos positivos y falsos negativos: un dominio recién registrado y aún no reportado por nadie devuelve “limpio” aunque sea malicioso (falso negativo por ventana temporal, ya discutido en la sección 1), mientras que un reporte erróneo o desactualizado puede marcar como malicioso un sitio que ya fue remediado (falso positivo). Por eso el `RiskEngine` las trata como una señal de alta confianza pero no como veredicto único: complementan, no reemplazan, el análisis heurístico y los modelos entrenados localmente.

5. Arquitectura de API REST con FastAPI

El backend expone su funcionalidad como una API REST construida con FastAPI, un framework de Python asíncrono elegido por tres razones: valida automáticamente el contrato de entrada y salida mediante modelos Pydantic (lo que impide, por ejemplo, que una URL sin prefijo `http`/`https` llegue a procesarse), genera de forma automática documentación interactiva (OpenAPI / Swagger) a partir del propio código, y soporta de forma nativa operaciones asíncronas y paralelas — esenciales para un pipeline que dispara seis llamadas de E/S en paralelo (WHOIS, HTML, tres APIs externas y un modelo de ML) por cada análisis.

El estilo arquitectónico REST se apoya en un conjunto de principios que el diseño de los once endpoints respeta de forma explícita: cada solicitud es independiente y no depende de un estado guardado en el servidor entre llamadas (statelessness) — el propio caché de resultados es una optimización de rendimiento, no un requisito de sesión —, y la interfaz es uniforme: los recursos (un análisis de URL, un análisis de contenido, una corrección de feedback) se identifican por ruta y se operan mediante verbos HTTP estándar (`POST` para crear un análisis, `GET` para

consultar metadatos o estadísticas, `DELETE` para vaciar el caché). A nivel de servidor, FastAPI corre sobre ASGI (Asynchronous Server Gateway Interface) en vez del más tradicional WSGI: WSGI atiende una solicitud por hilo y bloquea ese hilo mientras espera una operación de entrada/salida, mientras que ASGI permite que una sola tarea ceda el control mientras espera una respuesta de red y otra tarea avance mientras tanto. Esa diferencia es la que hace posible que el pipeline dispare WHOIS, la descarga de HTML y tres llamadas a APIs externas de forma concurrente dentro de un único worker, en vez de serializarlas una tras otra.

6. Seguridad de aplicaciones web expuestas a entrada no confiable

Como el backend recibe URLs arbitrarias controladas por el usuario y las descarga (`HtmlFetcher`), el diseño tuvo que incorporar defensas específicas contra Server-Side Request Forgery (SSRF): cada hostname se resuelve y se rechaza si alguna de sus IPs es privada, loopback, link-local o pertenece a un endpoint de metadatos de nube (169.254.169.254), las redirecciones se siguen manualmente re-validando el host en cada salto, y el cuerpo de la respuesta se limita a 2 MB. A esto se suman un límite de tasa por IP (`RateLimitMiddleware`, 30 solicitudes por minuto), un cortacircuitos (circuit breaker) que protege la cuota de VirusTotal, autenticación opcional por cabecera `X-API-Key` comparada con `hmac.compare_digest` para evitar ataques de temporización, y una política de CORS restringida por expresión regular a orígenes `chrome-extension://` y `localhost`.

Estas defensas responden a categorías reconocidas dentro del OWASP Top 10 (SSRF es, desde 2021, una categoría propia), y su combinación sigue el principio de defensa en profundidad: en vez de confiar en un único control que, si falla o se configura mal, deja el sistema completamente expuesto, se apilan varias capas independientes de modo que el fallo de una no compromete a las demás (la validación de esquema en Pydantic, la resolución y verificación de IP por hostname, el límite de tamaño de respuesta y el límite de tasa por IP operan en puntos distintos del pipeline). La comparación de la API key con `hmac.compare_digest` en vez del operador `==` responde a otra categoría de ataque menos conocida: un ataque de temporización (timing attack). Una comparación ingenua de cadenas se detiene en

el primer carácter que no coincide, de modo que una clave incorrecta que comparte más caracteres iniciales con la real tarda, en promedio, una fracción de microsegundo más en rechazarse; midiendo esa diferencia miles de veces, un atacante puede reconstruir el clave carácter por carácter sin necesidad de adivinarla completa de una sola vez. ``hmac.compare_digest`` compara en tiempo constante, independientemente de en qué posición difieran las cadenas, eliminando ese canal lateral de información.

El cortacircuitos (circuit breaker) que protege la cuota de VirusTotal aplica a una dependencia externa un patrón nacido en sistemas distribuidos: en vez de seguir llamando a una API que ya está devolviendo errores de cuota agotada (lo que solo desperdicia tiempo de espera y puede empeorar el problema), el circuito “se abre” tras detectar el agotamiento y deja de intentar llamadas durante una ventana de tiempo, permitiendo que el resto del pipeline siga respondiendo con las demás señales disponibles en vez de bloquearse esperando una respuesta que no va a llegar. Combinado con ``_safe()`` — que envuelve cada llamada paralela y convierte cualquier excepción en un resultado ``{"error": ...}`` en vez de propagarla — el efecto es que ninguna dependencia externa individual (una API caída, una cuota agotada, un WHOIS que no responde) puede tumbar el análisis completo: el peor caso es que esa señal específica quede ausente y el ``RiskEngine`` calcule el puntaje con las señales restantes, reflejándolo además en una confianza calibrada más baja.

7. Extensiones de navegador (Manifest V3)

Manifest V3 es el modelo de extensiones vigente en navegadores basados en Chromium. Reemplaza los scripts de fondo persistentes por service workers efímeros, prohíbe la ejecución de código remoto (todo el JavaScript debe empaquetarse dentro de la extensión) y exige declarar explícitamente cada permiso solicitado. El proyecto aprovecha ese modelo restrictivo a su favor: no solicita los permisos ``tabs`` ni ``activeTab``, por lo que nunca analiza una pestaña automáticamente — todo análisis es iniciado explícitamente por el usuario — y el ``background.js`` se limita a un chequeo periódico de salud del backend.

El cambio de Manifest V2 a V3 no fue cosmético: los scripts de fondo persistentes de V2 permanecían cargados en memoria de forma indefinida, consumiendo

recursos incluso cuando la extensión no se estaba usando activamente, y su ejecución continua ampliaba la superficie de ataque disponible para un script malicioso inyectado. Los service workers efímeros de V3 se cargan bajo demanda y se descargan cuando quedan inactivos, lo que mejora el rendimiento del navegador y reduce la ventana en la que código en segundo plano puede actuar sin que el usuario lo note. El modelo de permisos declarativos de V3 refuerza esa misma filosofía: cada permiso que una extensión necesita (acceso a pestañas, a almacenamiento, a un dominio remoto concreto) debe declararse explícitamente en el `manifest.json` y se muestra al usuario en el momento de la instalación, en vez de solicitarse de forma implícita o ampliarse dinámicamente en tiempo de ejecución. Junto con una Content Security Policy que impide cargar o ejecutar código remoto, esto limita de forma estructural lo que la extensión puede hacer incluso si parte de su código se viera comprometido.

8. Prácticas de ingeniería: contenedores, CI/CD y observabilidad

El backend se distribuye como imagen Docker (`python:3.12-slim`), que descarga los pesos de los tres modelos desde Hugging Face Hub durante la construcción de la imagen (build time), de modo que el contenedor final no depende de una descarga en tiempo de ejecución. La integración continua (GitHub Actions) ejecuta en cada push o pull request a `main` el linter (`ruff`) y la batería completa de pruebas automatizadas contra un backend con los modelos simulados (mockeados), de forma que la suite no necesita los archivos de peso reales para validar la lógica de negocio. Un segundo flujo de trabajo (`smoke-test.yml`) verifica, después de cada despliegue, que el endpoint `/predict` responde de verdad con un puntaje — porque `/health` y `/metadata` pueden devolver 200 incluso cuando la inferencia real está fallando por falta de memoria. La observabilidad local se apoya en Prometheus y Grafana, con métricas de conteo de solicitudes, histogramas de latencia y duración de arranque expuestas en `/metrics`.

Un contenedor difiere de una máquina virtual en el nivel al que ocurre la aislación: una máquina virtual virtualiza hardware completo y arranca su propio sistema operativo, mientras que un contenedor comparte el kernel del sistema anfitrión y aísla solo el proceso — su sistema de archivos, red y dependencias —, lo que lo

hace mucho más liviano y rápido de arrancar. Esa portabilidad es la que resuelve de raíz el problema clásico de “en mi máquina funciona”: la imagen construida en CI es exactamente la misma que corre en producción, con las mismas versiones de Python, de las librerías y de los pesos de los modelos, en vez de depender de que el entorno de destino tenga instalado por separado lo mismo que el entorno de desarrollo. La observabilidad del proyecto se apoya en las tres señales que la disciplina suele considerar complementarias entre sí: métricas (series numéricas agregadas en el tiempo — conteo de solicitudes, histogramas de latencia, expuestas vía Prometheus), logs (eventos discretos con contexto, como el `RequestLoggingMiddleware` que registra método, ruta, código de estado, latencia e IP de cada solicitud) y, aunque no se implementó un sistema de trazas distribuidas completo, la duración de arranque expuesta en `/metrics` cumple un rol equivalente al de una traza de un solo tramo: permite saber cuánto tiempo tomó cada fase de inicialización del servicio.

Análisis del problema:

El diagnóstico inicial del proyecto (sesión 1) identificó que un usuario común no dispone de una forma rápida de verificar la legitimidad de una URL antes de interactuar con ella: las defensas existentes viven del lado del proveedor de correo o del propio navegador, no son transparentes en sus razones, y no ofrecen al usuario una segunda opinión bajo demanda. Los riesgos iniciales identificados fueron tres: (1) depender de una sola fuente de verdad (por ejemplo, solo un modelo de ML) generaría falsos positivos y falsos negativos difíciles de auditar; (2) un backend que analiza URLs arbitrarias enviadas por un cliente no confiable (la extensión, distribuible por cualquiera) está expuesto a abuso (SSRF, agotamiento de cuota de APIs externas, scraping); y (3) los modelos de Machine Learning tienen un costo de memoria significativo, que en un entorno de despliegue gratuito y acotado (512 MB) podía volverse el cuello de botella del propio backend.

La deuda técnica del MVP inicial (2026-06-03) era considerable: el Random Forest inicial usaba solo 14 características, no existía fusión entre modelos, no había

integración con ninguna API de threat intelligence más allá de VirusTotal y Safe Browsing, la extensión no tenía panel lateral ni página de configuración, y no existía batería de pruebas automatizadas. La evolución documentada en el historial de cambios del proyecto muestra un camino incremental: incorporación del clasificador RoBERTa de URLs y del `FusionEngine` (junio), interfaz completa de la extensión con popup, sidebar y opciones (junio), corrección de un error crítico en el Random Forest que invertía en silencio las probabilidades de phishing/legítimo (afectando el 40 % del resultado de la fusión), incorporación del guardián SSRF, monitorización con Prometheus/Grafana y trazado con OpenTelemetry (agosto), calibración del motor de riesgo con probabilidad, intervalos de confianza y concordancia entre modelos (agosto), y finalmente la resolución de un fallo de memoria en producción causado, de forma contraintuitiva, por la cuantización int8 que se había introducido para acelerar la inferencia (agosto). Esa última corrección documentada con detalle en el registro de cambios del proyecto es un ejemplo directo de deuda técnica reconocida y resuelta con evidencia medida, no supuesta.

Propuesta de solución:

La solución adoptada separa el sistema en dos componentes independientes que se comunican por HTTP: una extensión de navegador (cliente, sin lógica de decisión propia) y un backend de análisis (donde vive toda la inteligencia). Esta separación se eligió sobre la alternativa de un modelo embebido en el propio navegador porque los modelos de ML usados (RoBERTa, en particular) son demasiado pesados para ejecutarse razonablemente dentro de una extensión, y porque mantener los modelos del lado del servidor permite actualizarlos sin republicar la extensión en cada tienda.

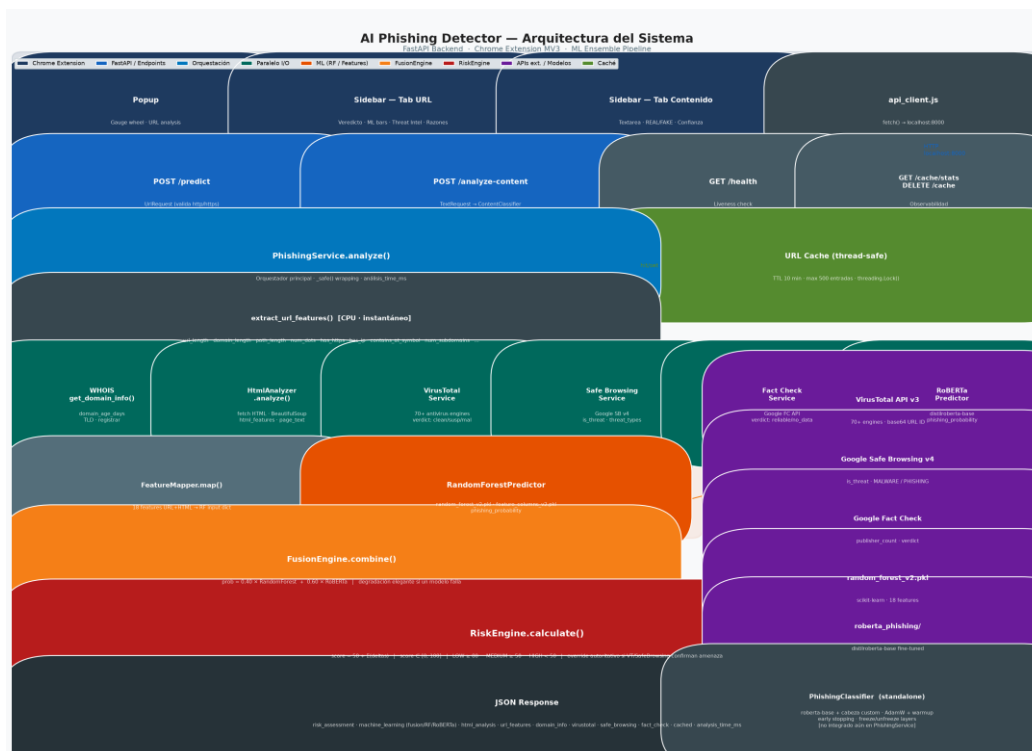


Figura 1. Arquitectura del pipeline de análisis (docs/architecture_diagram.png).

El pipeline de análisis (PhishingService.analyze) se organiza en dos oleadas paralelas seguidas de una etapa secuencial. La extracción de características de la URL (12 features, solo CPU) corre primero. Después, un “ThreadPoolExecutor” de 6 hilos ejecuta en paralelo: consulta WHOIS del dominio, descarga y análisis del HTML, consulta a VirusTotal, consulta a Google Safe Browsing, consulta a Google Fact Check y la inferencia del clasificador RoBERTa de URL. Una segunda etapa ejecuta el Random Forest, que depende del resultado del HTML de la etapa anterior. Finalmente, de forma secuencial, el FusionEngine combina ambos modelos de ML y el `RiskEngine` agrega todas las señales en un puntaje final con sus razones. El resultado se cachea en memoria durante 10 minutos para no repetir trabajo ni gastar cuota de API ante la misma URL.

Se eligió deliberadamente un motor de riesgo aditivo y explicable (RiskEngine) por encima de delegar la decisión final a un único modelo de caja negra, precisamente porque la evaluación exige poder explicar decisiones y reconocer riesgos: cada componente del puntaje final queda trazado a una señal concreta, auditable línea por línea en (docs/decision_tree.md).

Metodología de desarrollo del proyecto:

1. API inteligente y contratos

El backend expone 11 endpoints bajo un contrato tipado con modelos Pydantic tanto de entrada (UrlRequest, TextRequest) como de salida (un response_model distinto por endpoint, documentado en el esquema OpenAPI). Solo cuatro rutas son públicas; el resto exige la cabecera “X-API-Key” cuando el backend la tiene configurada.

Método	Ruta	Descripción	Auth
POST	/predict	Análisis completo de una URL (pipeline multi-señal)	Sí*
POST	/analyze-content	Clasificación de texto libre (REAL/FAKE)	Sí*
POST	/feedback	Registrar una corrección del usuario sobre un veredicto	Sí*
GET	/feedback/stats	Correcciones acumuladas (falsos positivos/negativos)	Sí*
GET	/experiment/status	Configuración activa del experimento de scoring	Sí*
GET	/cache/stats	Estadísticas del cache en memoria	Sí*
DELETE	/cache	Vaciar el cache	Sí*
GET	/	Comprobación básica de disponibilidad	No
GET	/health	Liveness check	No
GET	/metadata	Versión, modelos presentes y configuración activa	No
GET	/metrics	Métricas Prometheus (fuera del esquema OpenAPI)	No

*La autenticación es un no-op cuando `API\_KEY` está vacío. En este despliegue está deliberadamente vacío y declarado como público (`ALLOW\_UNAUTHENTICATED=true`), porque la extensión se distribuye por una tienda y su paquete es descomprimible por cualquiera: una clave embebida en el cliente sería pública desde el primer día. La protección real del backend público

recae en el límite de tasa por IP, el cortacircuitos de cuota de VirusTotal y el guardián SSRF.

Cada solicitud inválida (por ejemplo, una URL sin prefijo `http`/`https`) es rechazada por la validación de Pydantic con `422 Unprocessable Entity` antes de tocar cualquier lógica de negocio. Un manejador de excepciones global captura cualquier error no controlado, lo registra con su traza completa y devuelve `500` sin exponer detalles internos cuando el entorno es de producción.

2. Pruebas y CI/CD

La suite automatizada cuenta actualmente con 535 pruebas (pytest), repartidas entre “backend/tests/” y “tests/”, y corre sin necesidad de los archivos de modelo reales (“.pkl” ni directorios de Hugging Face): un “conftest.py” compartido sustituye los cargadores de modelo antes de que se importe el backend, y limpia entre pruebas el estado global del limitador de tasa y del cortacircuitos de VirusTotal.

Área cubierta	Qué valida
Guardián SSRF	34 pruebas → validación de IPs privadas/reservadas, prevención de DNS-rebinding
Límite de tasa	7 pruebas → cabeceras de proxy, bucketing por IP, evicción del store
Cortacircuitos VirusTotal	9 pruebas → agotamiento de cuota, reinicio, degradación controlada
Seguridad general	8 pruebas → filtrado de errores, ausencia de fuga de internals
Pipeline completo	200+ pruebas → features de URL/HTML, fusión, calibración, motor de riesgo

El flujo de integración continua (.github/workflows/ci.yml) instala las dependencias con la rueda de PyTorch para CPU (“requirements-dev.txt”, distinto del “requirements.txt” de producción que fija CUDA), ejecuta “ruff check .” y luego “pytest -v” en cada push o pull request contra “main”. Un segundo flujo (smoke-test.yml) se dispara después de cada despliegue y llama al “/predict” real del servicio publicado no a “/health” porque “/metadata” solo comprueba que los archivos de modelo existan en disco, sin decir nada sobre si la inferencia efectivamente cabe en memoria.

3. Despliegue e infraestructura

El backend se despliega como contenedor Docker en Render, a partir de ``backend/Dockerfile`` (imagen base ``python:3.12-slim``), con auto-deploy habilitado en cada push a ``main``. Durante la construcción de la imagen se descargan desde Hugging Face Hub (``mel3601/phishing-ia-models``) los tres modelos entrenados (~821 MB en total: Random Forest 25 MB, RoBERTa URL 317 MB, RoBERTa Content 479 MB), que quedan horneados en ``/models``. La carga de cada modelo en memoria es perezosa (``@lru_cache``), de modo que el primer ``/predict`` tras un reinicio paga el costo de carga (~10-20 s) y las solicitudes siguientes lo reutilizan.

Las variables de entorno relevantes (``VIRUSTOTAL_API_KEY``, ``SAFE_BROWSING_API_KEY``, ``FACT_CHECK_API_KEY``, ``API_KEY``, ``ALLOWED_ORIGINS``, ``ENVIRONMENT``, ``ALLOW_UNAUTHENTICATED``, ``MAX_CONCURRENT_ANALYSES``, ``ANALYSIS_QUEUE_TIMEOUT``, ``FORWARDED_ALLOW_IPS``) se configuran en el panel de Render y están documentadas, sin valores reales, en ``.env.example`` y en ``docs/DEPLOYMENT.md``. La reproducibilidad de la configuración no depende de pasos manuales no documentados: el Dockerfile y el flujo de despliegue son el mismo para cualquier colaborador que clone el repositorio.

Resultados alcanzados

1. Observabilidad, rendimiento y escalabilidad

El arranque en frío (cold start) del servicio, tras 15 minutos de inactividad en el plan gratuito de Render, toma entre 60 y 90 segundos: 5-10 s de arranque del contenedor, 10-20 s de carga perezosa de los modelos y ~5 s de arranque de FastAPI. Con los modelos ya cargados en memoria, una solicitud típica a ``/predict`` se descompone así:

Etapa	Duración aproximada
Extracción de características de URL	~100 ms
Descarga y análisis del HTML (si aplica)	+500-1500 ms
Inferencia de ML (en paralelo)	+200-500 ms

Etapas	Duración aproximada
Agregación del RiskEngine	~100 ms
Total típico	3-8 s (dominado por E/S de red, no por cómputo)

El cuello de botella identificado no fue la latencia sino la memoria: en el plan gratuito de Render (512 MB, un solo worker), dos análisis concurrentes hacían que el proceso superara el límite y el kernel lo terminara (``OOM-kill``), lo que además tumbaba ``/health`` junto con ``/predict`` (502 en producción). La causa raíz medida fue contraintuitiva: la cuantización int8 del modelo RoBERTa, introducida para acelerar la inferencia en CPU, en realidad materializaba todos los pesos del modelo en el proceso de conversión, generando un pico de +735 MB de RAM para ganar apenas 1,96 ms por URL en un pipeline de 3-8 s dominado por red. Al retirarla, el consumo de ``/predict`` bajó de 1213 MB a 791 MB. Como control adicional se añadió un semáforo de concurrencia (``MAX_CONCURRENT_ANALYSES``, por defecto 1) que serializa los análisis en vez de dejar que la memoria pico se multiplique por solicitudes simultáneas. El plan de escalabilidad para tráfico mayor al actual contempla, en orden de costo: subir a un plan de Render con más memoria, entrenar un Random Forest con menos árboles (reduciendo su huella de 156 MB) una vez exista un conjunto de validación etiquetado que justifique el compromiso con la precisión, y si el tráfico lo amerita separar el servicio en más de una instancia detrás de un balanceador.

La observabilidad local se implementa con Prometheus (``/metrics``, fuera del esquema OpenAPI) y un panel de Grafana provisionado por ``docker-compose.yml``, que exponen conteo de solicitudes por endpoint y código de estado (`http_requests_total`), histograma de latencia (`http_request_duration_seconds`) y duración de arranque (`app_startup_duration_seconds`). Cada solicitud queda además registrada por el ``RequestLoggingMiddleware`` con método, ruta, código de estado, latencia e IP de origen, lo que permite correlacionar un incidente puntual con su traza en los logs.

2. Seguridad, release y rollback

Riesgo	Control implementado
SSRF vía URL controlada por el usuario	Resolución y validación de IP por hostname, redirecciones re-validadas en cada salto, límite de 5 saltos y 2 MB de respuesta
Abuso / agotamiento de cuota de terceros	Límite de 30 solicitudes/60 s por IP; cortacircuitos dedicado para la cuota de VirusTotal
Comparación insegura de la API key	"hmac.compare_digest" en vez de "==", para evitar un canal lateral por temporización
Fuga de datos de navegación del usuario	Las URLs corregidas por el usuario se guardan hasheadas (SHA-256) en "feedback_store.py", nunca en claro
Fuga de detalles internos ante un error	El campo "detail" se omite en las respuestas de error cuando "ENVIRONMENT=production"
Arranque accidental sin autenticación	"ENVIRONMENT=production" sin "API_KEY" aborta el arranque, salvo declaración explícita "ALLOW_UNAUTHENTICATED=true"
Código remoto en la extensión (rechazado por las tiendas)	Manifest V3 lo prohíbe; se retiró un script cargado desde CDN que las tiendas habrían rechazado

Los casos adversariales se validan tanto en pruebas manuales documentadas (URL con IP en vez de dominio, URL con símbolo "@" para redirección engañosa, dominio con múltiples palabras clave de phishing, todos clasificados correctamente como HIGH) como en la suite automatizada del guardián SSRF. La verificación de humo post-despliegue (scripts/smoke_test.py) llama al "/predict" real, no al "/health", precisamente porque un backend con los modelos rotos puede seguir devolviendo 200 en los endpoints superficiales.

El release que se mostrará se quedó etiquetado como v1.0.0-VersiónFinal (GitHub Release, https://github.com/melsysdev-web/phishing_ia/releases/tag/v1.0.0-Versi%C3%B3nFinal), asociado al commit a8e2e39 de main el mismo estado de código descrito en todo este informe. Sus notas documentan qué incluye la versión y sus limitaciones conocidas, con trazabilidad directa al commit exacto.

El backend está desplegado públicamente en <https://verinews-sitio-web.netlify.app/>, correspondiente al release v1.0.0-VersiónFinal.

El mecanismo de rollback disponible es el propio de Render sobre despliegues por Git: cada despliegue queda asociado a un commit, y volver a una versión anterior implica re-desplegar el commit previo o revertirlo en “main”. Los límites operativos conocidos del sistema en este momento son: un único worker por instancia (WEB_CONCURRENCY=1), como máximo un análisis concurrente por defecto (MAX_CONCURRENT_ANALYSES=1), 512 MB de memoria disponible, y un límite de 30 solicitudes por minuto por IP compartido entre /predict y “/analyze-content”.

Conclusiones:

Nuestro proyecto entrega un sistema de detección de phishing funcional, explicable y desplegado, que combina aprendizaje automático, reglas heurísticas auditable e inteligencia de amenazas de terceros dentro de un pipeline que degrada con control ante el fallo de cualquiera de sus señales. Más allá del producto en sí, el desarrollo nos dejó aprendizajes concretos de ingeniería: que la optimización de inferencia (cuantización) puede tener el efecto contrario al esperado bajo restricciones de memoria si no se mide directamente el consumo pico; que exponer un backend a URLs arbitrarias enviadas por un cliente no confiable exige defensas SSRF explícitas y no un simple “requests.get;” y que empaquetar una extensión de navegador para una tienda tiene reglas de validación (ubicación del manifiesto, ausencia de código remoto) que no aparecen al probar la extensión sin empaquetar.

La limitación más relevante es de datos: no existe todavía un conjunto de validación etiquetado independiente contra el cual medir la precisión real de los modelos, por lo que las mejoras de scoring implementadas se concentran en calibración (que la confianza reportada refleje mejor la certeza real) y en reducir una fuente conocida de falsos positivos (el dominio recién registrado), en vez de en una cifra de accuracy que no podría respaldarse con evidencia. Un ensemble apilado y el fine-tuning adicional de RoBERTa sobre más datos son varias de las mejoras del plan original que quedaron deliberadamente sin implementar por la misma razón, y su desbloqueo depende de acumular correcciones reales de usuarios a través de “POST /feedback”, mecanismo que sí quedó construido y funcionando.

Otras limitaciones conocidas son: el backend de producción es intencionalmente público (sin autenticación real), porque una extensión distribuida en una tienda no puede resguardar un secreto; y la capacidad del plan gratuito de despliegue es acotada (un solo worker, un análisis concurrente).

Referencias:

- FastAPI — Documentación oficial. <https://fastapi.tiangolo.com/>
- scikit-learn — Random Forest Classifier. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- Hugging Face — Transformers y RoBERTa. https://huggingface.co/docs/transformers/model_doc/roberta
- Liu, Y. et al. — RoBERTa: A Robustly Optimized BERT Pretraining Approach (arXiv:1907.11692).
- Vaswani, A. et al. — Attention Is All You Need (arXiv:1706.03762).
- Google — Safe Browsing API (v4). <https://developers.google.com/safe-browsing>
- Google — Fact Check Tools API. <https://developers.google.com/fact-check/tools/api>
- VirusTotal — API v3 Documentation. <https://docs.virustotal.com/reference/overview>
- OWASP — Server-Side Request Forgery (SSRF) Prevention Cheat Sheet. <https://cheatsheetseries.owasp.org/>
- Chrome for Developers — Manifest V3. <https://developer.chrome.com/docs/extensions/mv3/intro/>
- Render — Docker Deploys Documentation. <https://render.com/docs/docker>
- Prometheus — Documentación oficial. <https://prometheus.io/docs/>

- Documentación interna del proyecto: docs/architecture.md, docs/decision_tree.md, docs/api.md, docs/DEPLOYMENT.md, docs/TESTING.md, SCORE_IMPROVEMENTS_STRATEGY.md.